

Assignment 4

Responsible Lecturer: Meni Adler

Responsible TA: Ofir Shahar

General Instructions

Submit your answers to the theoretical questions in a pdf file called ex4.pdf and your code for programming questions inside the provided ex4.rkt, ex4.pl files. ZIP those 3 files together into a file called id1_id2.zip.

Use the forum in a responsible manner so that the forum remains a useful resource for all students: do not ask questions that were already asked, limit your questions to clarification questions about the assignment, do not ask questions “is this correct”. We will not answer questions in the forum on the last day of the submission.

Question 1 - Lazy lists [25 points]

1.1 Theoretical [10 points, ex4.pdf]

- Define an equivalence criterion for two lazy lists (when do we say that two lazy lists are equivalent)
- Show that the following **even-squares-1** and **even-squares-2** lazy lists are equivalent according to your definition.

```
(define integers-from
  (lambda (n)
    (cons-lzl n (lambda () (integers-from (+ n 1))))))
```

```
(define lzl-map
  (lambda (f lzl)
    (if (empty-lzl? lzl)
        lzl
        (cons-lzl (f (head lzl))
                  (lambda () (lzl-map f (tail lzl)))))))
```

```
(define lzl-filter
  (lambda (p lzl)
    (cond ((empty-lzl? lzl) lzl)
```

```

((p (head lz)) (cons-lz (head lz)
                        (lambda () (lz-filter p (tail lz)))))
(else (lz-filter p (tail lz)))))

(define even-square-1
  (lz-filter (lambda (x) (= (modulo x 2) 0))
    (lz-map (lambda (x) (* x x))
      (integers-from 0))))

(define even-square-2
  (lz-map (lambda (x) (* x x))
    (lz-filter (lambda (x) (= (modulo x 2) 0))
      (integers-from 0))))

```

1.2 Practical [15 points, ex4.rkt]

In class, we implemented the procedure *sqrt*, which gets a number and returns its square root according to Newton's method:

```

(define sqrt (lambda (x init epsilon) (sqrt-iter x init epsilon)))

(define sqrt-iter
  (lambda (x guess epsilon)
    (if (good-enough? guess x epsilon)
        guess
        (sqrt-iter x (improve guess x) epsilon))))

(define abs (lambda (x) (if (< x 0) (- x) x)))
(define square (lambda (x) (* x x)))

(define good-enough?
  (lambda (guess x epsilon)
    (< (abs (- (square guess) x)) epsilon)))

(define average
  (lambda (x y) (/ (+ x y) 2)))

(define improve
  (lambda (guess x)
    (average guess (/ x guess))))

```

```
(sqrt 2 1 0.0001)
```

```
→
```

```
577/408
```

In this version, the *improve-guess* step is repeated until a ‘good-enough’ approximation is obtained. In contrast, the procedure *sqrt-lzl* returns **all** guesses, including those which are not ‘good-enough’ yet, and those which are beyond the ‘good-enough’ criterion.

The procedure *sqrt-lzl*, gets a number and a non-zero initial guess for its square root, and returns a lazy list composed of all its approximated roots, each represented by (guess . accuracy) pairs. The ‘accuracy’ value at each pair indicates the proximity of the given guess to the root of the given number x : $(\text{abs } (- (\text{square guess}) x))$

For example:

```
(take (sqrt-lzl 2 1) 5)
```

```
→
```

```
'((1 . 1) (3/2 . 1/4) (17/12 . 1/144) (577/408 . 1/166464) (665857/470832 . 1/221682772224))
```

- a. Implement the *sqrt-lzl* procedure.

Signature: *sqrt-lzl*(x , *init*)

Purpose: Generate a lazy list of approximations (pairs of <guess, accuracy>) of the square root of the given number x , according to Newton method, starting from *init* guess.

Type: [Number * Number -> LzlList<Pair<Number,Number>>]

Pre-condition: *init* $\neq 0$

Tests: (take (*sqrt-lzl* 2 1) 3) → '((1 . 1) (3/2 . 1/4) (17/12 . 1/144))

- b. Implement the procedure *find-first* which gets a lazy list and a predicate, and returns the first item in the list which satisfies the predicate. In case the item was not found (for the case of finite lazy list) return ‘fail’.

```
(find-first (integers-from 1) (lambda (x) (> x 10)))
```

```
→
```

```
11
```

```
(find-first (cons-lzl 1 (lambda() (cons-lzl 2 (lambda () '())))) (lambda (x) (> x 10)))
```

```
→
```

```
'fail
```

Signature: *find-first*(*lzlst*, *p*)

Purpose: Return the first item in the given lazy list which satisfies the given predicate.

```

Type: [[LzlList<T> * T->Boolean] -> T | {'fail'} ]
Pre-condition: /
Tests: (find-first (integers-from 1) (lambda (x) (> x 10))) →
11; ;;Tests: (find-first (integers-from 1) (lambda (x) (> x
10))) → 11; (find-first (cons-lzl 1 (lambda() (cons-lzl 2
(lambda () '())))) (lambda (x) (> x 10))) → 'fail

```

- c. Implement the procedure `sqrt2`, which gets the same parameters of the above `sqrt` procedure and returns the same value, by applying *sqrt-lzl* and *find-first*: it should first generate the lazy list of all approximations, and then select the first approximation which fit the 'epsilon' measure.

```

(sqrt2 2 1 0.0001)
→
577/408

```

```

Signature: sqrt2(x,init,epsilon)
Purpose: return approximation of the square root of the given
number x, according to Newton method, starting from init guess
with epsilon threshold. The procedure uses sqrt-lzl and
find-first procedures.
Type: [Number * Number * Number -> Number]
Pre-condition: init /= 0
Tests: (sqrt2 2 1 0.0001) → 577/408

```

Question 2 - CPS [25 points]

- a. Define an equivalence criterion for a procedure and its Success-Fail-Continuations version (when do we say that a procedure and its Success-Fail-Continuations version are equivalent). It helps to specify the type of the function to express this criterion. **[5 points, ex4.pdf]**
- b. Write a value-retrieval procedure **get-value** for association lists: **[5 points, ex4.rkt]**

```

Signature: get-value(assoc-list, key)
Purpose: Find the value of 'key'. If 'key' is not found, return
'fail.
Type: [List<Pair(Symbol,T)> * Symbol -> T | 'fail]
Tests: (get-value '((a . 3) (b . 4)) 'b) → 4, (get-value '((a .
3) (b . 4)) 'c) → 'fail

```

- c. Write a Success-Fail-Continuations version of get-value: **[5 points, ex4.rkt]**

Signature: `get-value$(assoc-list, key, success, fail)`
 Purpose: Find the value of 'key'. If 'key' is found, then apply the continuation 'success' on its value val. Otherwise, apply the continuation 'fail'.
 Type: `[ List<Pair<Symbol,T>> * Symbol * [T->T1] * [Empty->T2] ] -> T1 | T2`
 Tests: `> (get-value$ '((a . 3) (b . 4)) 'b (lambda(x) (* x x)) (lambda()#f)) → 16, (get-value$ '((a . 3) (b . 4)) 'c (lambda(x) (* x x)) (lambda()#f)) → #f`

- d. Show that get-value and get-value\$ are equivalent according to your definition. **[5 points, ex4.pdf]**
- e. The procedure collect-all-values gets a list of associated lists and a key, and returns a list composed of the values for this key in the various associated lists. **[5 points, ex4.rkt]**

Signature: `collect-all-values(list-assoc-lists, key)`
 Purpose: Returns a list of all values of the first occurrence of 'key' in each of the given association lists. If there's no such value, return the empty list.
 Type: `[List<List<Pair<Symbol,T>>> * Symbol -> List<T>]`
 Tests:
`> (define l1 '((a . 1) (e . 2)))`
`> (define l2 '((e . 5) (f . 6)))`
`> (collect-all-values (list l1 l2) 'e) → '(2 5)`
`> (collect-all-values (list l1 l2) 'k) --> '()`

Implement two versions of collect-all-values:

collect-all-values-1 - based on **get-value**

collect-all-values-2 - based on **get-value\$**

Question 3 - Logic programing [50 points]

3.1 Unification [15 points, ex4.pdf]

What is the result of these operations? Provide algorithm steps, and explain in case of failure.

1. `unify[t(s(s), G, s, p, t(K), s), t(s(G), G, s, p, t(K), U)]`

2. `unify[g(l,M,g,G,U,g,v(M)), g(l,v(U),g,v(M),v(G),g,v(M))]`

```

3. unify[m(M,N), n(M,N)]

4. unify[p([v | [V | VV]]), p([v | V] | VV)]

5. unify[g([T]), g(T)]

```

3.2 Logic programming [20 points, ex4.pl]

You cannot use any Prolog built-in procedures.

You can define your own procedure (including those implemented in class).

a. Write a procedure `sub_list(Sublist, List)/2` that defines following relation between two lists:

```

% Signature: sub_list(Sublist, List)/2
% Purpose: All elements in Sublist appear in List in the same order.
% Precondition: List is fully instantiated
% (queries do not include variables in their second argument).
% Example:
% ?- sub_list(X, [1, 2, 3]).
% X = [1, 2, 3];
% X = [1, 2];
% X = [1, 3];
% X = [2, 3];
% X = [1];
% X = [2];
% X = [3];
% X = [];
% false

```

B.

Read about Prolog's *cut* -'!'.

Write a procedure `swap_list(List, InversedList)/2` which defines the following relation between two lists:

```

% Signature: swap_list(+List, ?InversedList)/2
% Purpose: InversedList is the 'mirror' representation of List, i.e,
each item in the list is recursively replaced with the item at the
position, with refers to the beginning and the end of the list.
% Example:
% ?- swap_list([a,b,c,d,e],T).

```

```
% T = [e,d,c,b,a];
% ?- swap_list([a,b],[c,d],e,T).
% T = [e,[d,c],[b,a]];
```

You can use Prolog's cut for this SEIF (only)

c. Write a procedure `sub_tree(Subtree, Tree)/2` that defines following relation between two binary trees:

```
% Signature: sub_tree(?Subtree, +Tree)/2
% Purpose: Tree contains Subtree.
% Example:
% ?- sub_tree(X, tree(a, tree(b,void,void), tree(c,void,void))).
% X = tree(b,void,void);
% X = tree(c,void,void);
% X = tree(a, tree(b,void,void), tree(c,void,void));
```

d. Write a procedure `swap_tree(Tree, InversedTree)/2` that defines the following relation between two binary trees:

```
% Signature: swap_tree(+Tree, ?InversedTree)/2
% Purpose: InversedTree is the 'mirror' representation of Tree.
% Example:
% ?- swap_tree(tree(4,tree(2,tree(1,void,void),
tree(3,void,void)),tree(5,void,void)),T).
% T =
tree(4,tree(5,void,void),tree(2,tree(3,void,void),tree(1,void,void)))
;
```

3.3 Proof tree [15 points, ex4.pdf]

Unary numbers provide symbolic representation for the natural numbers. They are defined inductively as follows: zero is the atom `[]`, and for a Unary number `c`, `[1|c]` represents the number `c+1`. The numbers 0, 1, 2, 3 etc. are represented by the terms `[]`, `[1]`, `[1, 1]`, `[1, 1, 1]`, and so forth. The following program is given.

```
% Signature: unary_number(L)/1
% Purpose: L is a unary number
unary_number([]).           %1
unary_number([x|A]) :-      %2
    unary_number(A).
```

```

% Signature: unary_plus(X,Y,Z)/3
% Purpose: X append Y = Z
unary_plus(X, [], X) :- %1
    unary_number(X).
unary_plus(X, [A|Y], [A|Z]) :- %2
    unary_plus(X, Y, Z).

```

- a. Draw the proof tree for the query below and the given program. For success leaves, calculate the substitution composition and report the answer at each success leaf.

?- unary_plus ([1|X], [1,1|Y], [1,1,1|X]).

[8 points]

- b. Is this a success or a failure proof tree? Is this tree finite or infinite?

[2 points]

- c. Is this query provable from the given program?

[2 points]

- d. Let us define L9 as the *rational logic programming* language with the addition of one functor only - *cons*. Is L9 decidable?

[3 points]